FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Geofrey Miguel Erlano
Section:	AN41
Professor:	Ms. Crisola Tan
Date:	October 3, 2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base

2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
   The Rule-Based Reasoning (RBR) system was significantly easier to implement. Its architecture relies on deterministic conditional logic (IF-THEN statements), which maps directly to fundamental programming control structures. Developing the inference engine simply required looping through static conditions, whereas Case-Based Reasoning (CBR) necessitated the development of a more complex algorithm to calculate similarity scores and manage the retrieval, revision, and retention of data dictionaries.

2. Which is more flexible in solving unstructured problems?
   Case-Based Reasoning (CBR) demonstrates far greater flexibility when addressing unstructured or ambiguous problems. RBR systems are rigid; they require exact feature matches to trigger a rule and will fail to yield a result if an unanticipated scenario arises. Conversely, CBR utilizes heuristic similarity matching, allowing the system to extrapolate and propose highly probable solutions

based on the closest historical match, even when the current input parameters are incomplete or unprecedented.

3. Which is better for situations with continuously changing data?
   CBR is decidedly superior in environments characterized by continuously changing data. Through its inherent "Retain" phase, a CBR system autonomously updates its knowledge base by appending newly resolved cases, allowing it to adapt and improve over time without structural modification. In contrast, updating an RBR system to accommodate new variables requires manual intervention by a programmer or domain expert to continuously author and integrate new logical rules into the codebase.

4. Examples of real-world applications for both.
   Rule-Based Reasoning (RBR): Frequently applied in domains requiring strict adherence to predefined protocols, such as automated tax calculation software, banking compliance systems, and basic credit card fraud detection algorithms (e.g., flagging transactions that exceed specific geographic or monetary thresholds).

   Case-Based Reasoning (CBR): Utilized in complex, highly variable fields such as medical diagnostic support systems (where physicians compare novel patient symptoms against historical clinical cases) and advanced IT service management platforms (where helpdesk systems suggest hardware resolutions based on patterns in previously resolved support tickets).

Your Code - Laboratory Tasks ⇩

```
"""
Technical Assessment: KR, RBR, and CBR
Use Case: IT Helpdesk / AI Diagnostics System
Upgrades: Object-Oriented Design, Type Hinting, Dataclasses, and Normalized Similarity Scoring.
"""

from typing import List, Dict, Any, Callable
from dataclasses import dataclass

# ========================================
# PART 1 — Knowledge Representation (KR)
# ========================================

@dataclass
class LaptopState:
    """Task 1A: Knowledge Base represented as a strictly typed Data Class."""
```

```
    def retrieve_and_reuse(self, new_problem: Dict[str, Any]) -> str:
        """Task 3C: Advanced Similarity Matching (Normalized Jaccard-style Scoring)."""
        best_match = None
        highest_score = -1.0

        for case in self.case_base:
            # Calculate how many features match exactly
            matches = sum(1 for key, val in new_problem.items()
                          if key in case.features and case.features[key] == val)

            # Normalize score between 0.0 and 1.0 (Percentage match)
            score = matches / len(new_problem) if len(new_problem) > 0 else 0
```

```
    battery_level: int       # Category: Status
    cpu_temperature: int     # Category: Temperature
    screen_is_black: bool    # Category: Status
    power_on: bool           # Category: Presence
    strange_noises: bool     # Category: Presence

    def to_dict(self) -> Dict[str, Any]:
        """Converts the object to a dictionary for engine processing."""
        return self.__dict__

# Initialize our facts
current_laptop_state = LaptopState(
    battery_level=5,
    cpu_temperature=92,
    screen_is_black=True,
    power_on=False,
    strange_noises=False
)

#
========================================
# PART 2 — Rule-Based Reasoning (RBR)
#
========================================

class Rule:
    """Encapsulates a single rule with a lambda/function condition and an action."""
    def __init__(self, name: str, condition: Callable[[Dict], bool], action: str):
        self.name = name
        self.condition = condition
        self.action = action

class RuleBasedEngine:
    """Task 2B: An OOP Inference Engine."""
    def __init__(self):
        self.rules: List[Rule] = []

    def add_rule(self, rule: Rule):
        self.rules.append(rule)

    def evaluate(self, facts: Dict[str, Any]) -> List[str]:
        """Task 2C: Checks each rule and returns all inferred actions."""
        triggered_actions = []
        for rule in self.rules:
            if score > highest_score:
                highest_score = score
                best_match = case

        confidence = round(highest_score * 100, 2)
        return f"{best_match.solution} (Confidence: {confidence}%)" if best_match else "No match."

    def revise(self, suggested: str, actual_fix: str = "") -> str:
        """Task 3D: Revise Step."""
        return actual_fix if actual_fix.strip() else suggested

    def retain(self, new_problem: Dict[str, Any], final_solution: str):
        """Task 3E: Retain Step."""

self.case_base.append(Case(features=new_problem, solution=final_solution))

#
========================================
# MAIN EXECUTION
#
========================================
if __name__ == "__main__":
    print("="*60)
    print("🛠️ PART 2: RULE-BASED REASONING (RBR) DIAGNOSTICS")
    print("="*60)

    facts_dict = current_laptop_state.to_dict()
    print(f"SYSTEM TELEMETRY: {facts_dict}\n")

    actions = rbr_engine.evaluate(facts_dict)
    print("RBR INFERRED ACTIONS:")
    if actions:
        for action in actions:
            print(f" ➔ {action}")
    else:
        print(" ➔ System Normal. No actions required.")

    print("\n" + "="*60)
    print("🧠 PART 3: CASE-BASED REASONING (CBR) ENGINE")
    print("="*60)

    # Initialize CBR Engine
```

```
        if rule.condition(facts):
                triggered_actions.append(f"[{rule.name}] {rule.action}")
        return triggered_actions

# Task 2A: Create Rules dynamically
rbr_engine = RuleBasedEngine()
rbr_engine.add_rule(Rule(
    name="DeadBatteryRule",
        condition=lambda f: not f["power_on"] and f["battery_level"] < 10,
    action="Plug in the laptop charger immediately."
))
rbr_engine.add_rule(Rule(
    name="ThermalThrottlingRule",
    condition=lambda f: f["cpu_temperature"] > 85,
     action="Critical Temp: Shut down and clean cooling fans."
))
rbr_engine.add_rule(Rule(
    name="DisplayFailureRule",
          condition=lambda f: f["power_on"] and f["screen_is_black"],
     action="Connect to an external monitor to isolate GPU/Screen failure."
))

#
========================================
# PART 3 — Case-Based Reasoning (CBR)
#
========================================

@dataclass
class Case:
    features: Dict[str, Any]
    solution: str

class CaseBasedEngine:
     """A modular CBR system encapsulating the 4R Cycle."""
    def __init__(self):
        self.case_base: List[Case] = []

    def load_initial_cases(self, cases: List[Dict]):
        """Task 3A: Build a Case Base."""
        for c in cases:
```

```
    cbr_engine = CaseBasedEngine()
    initial_cases = [
        {"problem": {"power": False, "noise": False, "spill": False}, "solution": "Check wall outlet."},
        {"problem": {"power": False, "noise": True, "spill": False}, "solution": "Replace Power Supply Unit (PSU)."},
        {"problem": {"power": False, "noise": False, "spill": True}, "solution": "Liquid damage: Do not turn on. Disconnect battery."}
    ]
    cbr_engine.load_initial_cases(initial_cases)

    # Unstructured New Problem
     new_ticket = {"power": False, "noise": True, "spill": False}
    print(f"NEW TICKET OPENED: {new_ticket}\n")

    # 4R Cycle Execution
                                        suggestion = cbr_engine.retrieve_and_reuse(new_ticket)
    print(f"[1 & 2. RETRIEVE & REUSE] AI Suggestion: \n ↳ {suggestion}\n")

    # Simulating technician finding a different root cause
     tech_notes = "PSU fan was jammed by a wire. Cleared obstruction. PSU functional."
            final_fix = cbr_engine.revise(suggestion, actual_fix=tech_notes)
    print(f"[3. REVISE] Technician Override Applied: \n ↳ {final_fix}\n")

    cbr_engine.retain(new_ticket, final_fix)
     print(f"[4. RETAIN] Knowledge base updated. Total Cases: {len(cbr_engine.case_base)}")
    print("="*60)
```

self.case_base.append(Case(features=c["problem"], solution=c["solution"]))	